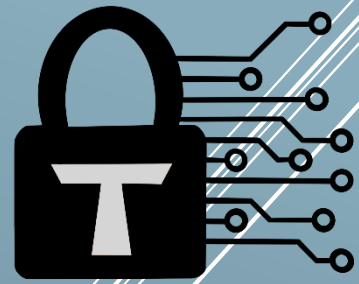


Trust Security



Smart Contract Audit

Reserve Trusted Fillers -
ImmutableTokenJar

24/01/2026

Executive summary

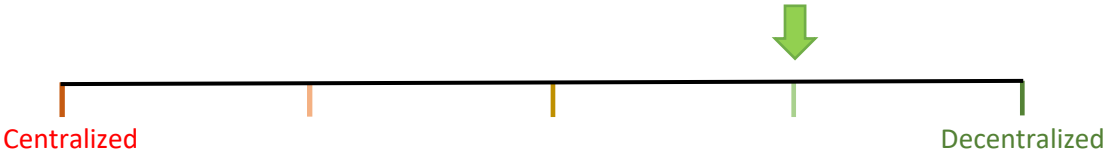


Category	Trading
Audited file count	1
Lines of code	57
Auditor	HollaDieWaldfee cccz
Time period	15/01/2026 – 24/01/2026

Findings

Severity	Total	Fixed
High	0	0
Medium	1	1
Low	1	1

Centralization score



Signature

EXECUTIVE SUMMARY	1
DOCUMENT PROPERTIES	3
Versioning	3
Contact	3
INTRODUCTION	4
Scope	4
Repository details	4
About Trust Security	4
About the Auditors	4
Disclaimer	4
Methodology	5
QUALITATIVE ANALYSIS	6
FINDINGS	7
Medium severity findings	7
TRST-M-1 Owner's signature in <i>ImmutableTokenJar</i> can be replayed in CoW Swap	7
Low severity findings	8
TRST-L-1 Orders can't be invalidated due to missing integration with <i>GPv2Settlement.invalidateOrder()</i>	8
Additional recommendations	9
TRST-R-1: Document systemic risks of CoW Swap trades	9
Centralization risks	10
TRST-CR-1: Owner in permissioned mode is fully trusted	10
Systemic risks	11
TRST-SR-1: Integration with CoW Swap	11

Document properties

Versioning

Version	Date	Description
0.1	24/01/2026	Client report

Contact

Trust

trust@trust-security.xyz

Introduction

Trust Security has conducted an audit at the customer's request. The audit is focused on uncovering security issues and additional bugs contained in the code defined in scope. Some additional recommendations have also been given when appropriate.

Scope

- `contracts/extras/ImmutableTokenJar.sol`

Repository details

- **Repository URL:** <https://github.com/reserve-protocol/trusted-fillers>
- **Commit hash:** 309401fe327f1a6799d49f2b7964887c0d0192cb
- **Mitigation hash:** 0facc4dc9f55bacd3c5447ababedd5dd37d6493e

About Trust Security

Trust Security has been established by top-end blockchain security researcher Trust, in order to provide high quality auditing services. Trust is the leading auditor at competitive auditing service Code4rena, reported several critical issues to Immunefi bug bounty platform and is currently a Code4rena judge.

About the Auditors

HollaDieWaldfee is a distinguished security expert with a track record of multiple first places in competitive audits. He is a Lead Auditor at Trust Security and Senior Watson at Sherlock.

A top competitor in audit contests, cccz has achieved superstar status in the security space. He is a Block Hat / DEFCON speaker with rich experience in both traditional and blockchain security.

Disclaimer

Smart contracts are an experimental technology with many known and unknown risks. Trust Security assumes no responsibility for any misbehavior, bugs or exploits affecting the audited code or any part of the deployment phase.

Furthermore, it is known to all parties that changes to the audited code, including fixes of issues highlighted in this report, may introduce new issues and require further auditing.

Methodology

In general, the primary methodology used is manual auditing. The entire in-scope code has been deeply looked at and considered from different adversarial perspectives. Any additional dependencies on external code have also been reviewed.

Qualitative analysis

Metric	Rating	Comments
Code complexity	Excellent	Project kept code as simple as possible, reducing attack risks.
Documentation	Good	Project is sufficiently documented.
Best practices	Excellent	Project follows best practices.
Centralization risks	Good	<i>ImmutableTokenJar</i> can be used in permissioned and permissionless mode.

Findings

Medium severity findings

TRST-M-1 Owner's signature in *ImmutableTokenJar* can be replayed in CoW Swap

- **Category:** Signature issues
- **Source:** ImmutableTokenJar.sol
- **Status:** Fixed

Description

In the permissioned mode, orders require a [signature](#) by the jar's **owner** over the EIP712 **orderHash**. However, this signature can also be used inside [CoW Swap](#) to execute a trade from the **owner** itself. Thus, instead of trading the tokens from the jar, the **owner**'s own tokens may be traded and sent to the jar. This, of course, requires that the **owner** has also approved **GPV2_VAULT_RELAYER**.

Signature re-use across jar deployments is not possible since **order.receiver** must be [address\(this\)](#), acting as an implicit domain separator.

Recommended mitigation

Signatures by the jar's **owner** must not be valid in contexts other than the specific jar. The most robust, yet verbose, solution is to implement EIP712 in *ImmutableTokenJar*. Alternatively, a more lightweight scheme may be adopted like signing a hash of the **orderHash**, i.e., a double-hash of the order.

Team response

Fixed in commit [6b1fd66](#).

Mitigation review

Verified, the issue has been addressed by requiring an **eth_sign** signature over the hashed [orderHash](#). Thus, the data signed for the jar becomes distinct and the signature can't be used inside CoW Swap.

Low severity findings

TRST-L-1 Orders can't be invalidated due to missing integration with *GPv2Settlement.invalidateOrder()*

- **Category:** Logical issues
- **Source:** ImmutableTokenJar.sol
- **Status:** Fixed

Description

CoW Swap's *GPv2Settlement* contract provides an [invalidateOrder\(\)](#) function that allows the **owner** of an order, i.e., the *ImmutableTokenJar*, to invalidate the order by setting its **filledAmount** to **type(uint256).max**. Since *ImmutableTokenJar* does not integrate with this *invalidateOrder()* function, orders signed by the jar's **owner** can only become invalid when they expire.

Recommended mitigation

It is recommended to expose *GPv2Settlement.invalidateOrder()* in *ImmutableTokenJar* and make it accessible to the **owner** of the jar.

Team response

Fixed in commit [b71d5b0](#).

Mitigation review

Verified, the recommendation has been implemented.

Additional recommendations

TRST-R-1: Document systemic risks of CoW Swap trades

ImmutableTokenJar, particularly in the permissionless mode where no prices are enforced by the jar, relies heavily on CoW Swap's solver competition. Furthermore, without **appData** validation, the jar is exposed to a known risk with regards to partner fees. It is recommended to document the following three systemic risks:

1. Trading tokens with insufficient liquidity / solver integration can weaken competition and solvers' ability to route efficiently.
2. Solvers [optimize a "score"](#), and while the high-level objective of this approach is to provide the best possible order execution, the complexity of the low-level details may allow for reward hacking strategies. Orders in the permissionless mode are especially vulnerable since they don't enforce any prices. CoW Swap actively punishes [malicious solver behavior and attempts to game the competition rules](#).
3. In the permissionless mode, **appData** is not validated for [hooks](#) to be dynamically inserted into the order. But this allows for up to **1%** of order volume to be extracted as partner fees. This risk is [documented](#) inside CoW Swap.

Centralization risks

TRST-CR-1: Owner in permissioned mode is fully trusted

In the permissioned mode, i.e., when **owner** is not **address(0)**, all orders must be signed by the **owner**. Consequently, if the **owner** becomes unavailable, intentionally or unintentionally, no trades can occur and tokens remain stuck in the contract.

Systemic risks

TRST-SR-1: Integration with CoW Swap

ImmutableTokenJar integrates with [CoW Swap](#) to execute intent-based trades. As such, this external dependency is fully trusted. [TRST-R-1](#) describes associated systemic risks in more detail.